

Trabajo Práctico N° 2: Sockets TCP vs UDP

Materia: Sistemas Distribuidos y Paralelos

Institución: Universidad Nacional de Villa Mercedes

Alumno: Barboza Luca, Faba Mizrahi Valentin

1. Resultados de la Prueba A (Funcionamiento normal)

Prueba A	TCP	UDP
Mensajes enviados	10	10
Respuestas válidas recibidas	10	10
RTT promedio (ms)	0.1188 ms	0.122 ms

2. Resultados de la Prueba B (Interrupción y comparación)

Aspecto	TCP	UDP
¿Requiere establecer una conexión?	Sí, mediante <code>connect()</code> y <code>accept()</code> antes de enviar datos[cite: 1, 2].	No, envía datagramas independientes sin conexión previa[cite: 1, 2].
Funciones de envío y recepción	<code>sendall()</code> y <code>recv()</code> .	<code>sendto()</code> y <code>recvfrom()</code> [cite: 2].
¿Conserva los límites de cada mensaje?	No, es un flujo continuo de bytes; requiere un delimitador (ej. <code>\n</code>) [cite: 1, 2].	Sí, conserva las fronteras de cada datagrama enviado[cite: 1, 2].
Comportamiento al detener el servidor	El cliente detecta la falla inmediatamente, lanza un error (o recibe bytes vacíos) y detiene la ejecución de manera abrupta[cite: 2].	El cliente no se interrumpe; registra la pérdida por expiración del timeout y continúa intentando enviar los siguientes mensajes[cite: 2].
Acciones para recuperar el intercambio	Es necesario reiniciar el servidor y volver a ejecutar el cliente desde cero para establecer una nueva conexión[cite: 2].	Solo se necesita reiniciar el servidor; el cliente en ejecución retoma la recepción automáticamente en los mensajes siguientes[cite: 2].

3. Respuestas a preguntas de observación

1. ¿Recibir todas las respuestas UDP demuestra que UDP garantiza la entrega? ¿Por qué?
No, no lo demuestra. UDP no garantiza la entrega ni el orden de los datagramas[cite: 1, 2]. El éxito en la recepción de todos los mensajes se debe a que la prueba se ejecutó localmente

utilizando la dirección loopback (127.0.0.1), en un entorno ideal sin congestión ni pérdida de paquetes en una red física real[cite: 1, 2].

2. ¿Qué protocolo elegirían para transferir un archivo? ¿Y para enviar periódicamente la temperatura de un sensor? Justifiquen.

- **Transferencia de archivo:** Se elige TCP porque provee una entrega confiable, garantiza el orden estricto de los bytes y maneja la retransmisión si hay errores, asegurando la integridad del archivo[cite: 1, 2].
- **Temperatura de sensor:** Se elige UDP porque tiene menor sobrecarga y latencia inicial (no requiere conexión)[cite: 1, 2]. En telemetría periódica, si se pierde una medición aislada no afecta el sistema, ya que la siguiente medición actualizará el estado rápidamente[cite: 1, 2].

3. ¿Por qué los tiempos medidos en una sola computadora no permiten afirmar que un protocolo siempre es más rápido?

Porque las mediciones locales evitan el paso por una red física real[cite: 1, 2]. No se toma en cuenta el enrutamiento IP a través de Internet, la latencia de los equipos de red, ni los mecanismos de TCP para el control de congestión y retransmisión, los cuales influyen fuertemente en el rendimiento final al comunicar computadoras diferentes[cite: 1, 2].

4. Salidas de Prueba (Capturas de consola)

Prueba A: Funcionamiento normal (TCP y UDP)

```
PS C:\Users\valen\OneDrive\Escritorio\Documentos\Facultad\4to año 2do cutrimestre\Sistemas Distribuidos Y Paralelos\tp2> python cliente_udp.py
Enviado/Recibido: Mensaje 1 | RTT: 0.445 ms
Enviado/Recibido: Mensaje 2 | RTT: 0.188 ms
Enviado/Recibido: Mensaje 3 | RTT: 0.414 ms
Enviado/Recibido: Mensaje 4 | RTT: 0.186 ms
Enviado/Recibido: Mensaje 5 | RTT: 0.185 ms
Enviado/Recibido: Mensaje 6 | RTT: 0.185 ms
Enviado/Recibido: Mensaje 7 | RTT: 0.245 ms
Enviado/Recibido: Mensaje 8 | RTT: 0.417 ms
Enviado/Recibido: Mensaje 9 | RTT: 0.230 ms
Enviado/Recibido: Mensaje 10 | RTT: 0.200 ms
```

```
PS C:\Users\valen\OneDrive\Escritorio\Documentos\Facultad\4to año 2do cutrimestre\Sistemas Distribuidos Y Paralelos\tp2> python cliente_tcp.py
Enviado/Recibido: Mensaje 1 | RTT: 0.420 ms
Enviado/Recibido: Mensaje 2 | RTT: 0.267 ms
Enviado/Recibido: Mensaje 3 | RTT: 0.501 ms
Enviado/Recibido: Mensaje 4 | RTT: 0.149 ms
Enviado/Recibido: Mensaje 5 | RTT: 0.332 ms
Enviado/Recibido: Mensaje 6 | RTT: 0.388 ms
Enviado/Recibido: Mensaje 7 | RTT: 0.193 ms
Enviado/Recibido: Mensaje 8 | RTT: 0.493 ms
Enviado/Recibido: Mensaje 9 | RTT: 0.184 ms
Enviado/Recibido: Mensaje 10 | RTT: 0.168 ms
```

Prueba B: Interrupción del servidor (TCP y UDP)

```
PS C:\Users\valen\OneDrive\Escritorio\Documentos\Facultad\4to año 2do cutrimestre\Sistemas Distribuidos Y Paralelos\tp2> python cliente_udp.py
Enviado/Recibido: Mensaje 1 | RTT: 0.505 ms
Enviado/Recibido: Mensaje 2 | RTT: 0.194 ms
Enviado/Recibido: Mensaje 3 | RTT: 0.279 ms
Traceback (most recent call last):
  File "C:\Users\valen\OneDrive\Escritorio\Documentos\Facultad\4to año 2do cutrimestre\Sistemas Distribuidos Y Paralelos\tp2\cliente_udp.py", line 24, in <module>
    time.sleep(2) # Pausa entre intercambios, fuera de la medición del RTT[cite: 2]
    ~~~~~^~~~~~
KeyboardInterrupt
```

```
PS C:\Users\valen\OneDrive\Escritorio\Documentos\Facultad\4to año 2do cutrimestre\Sistemas Distribuidos Y Paralelos\tp2> python cliente_tcp.py
Enviado/Recibido: Mensaje 1 | RTT: 0.220 ms
Enviado/Recibido: Mensaje 2 | RTT: 0.100 ms
Enviado/Recibido: Mensaje 3 | RTT: 0.187 ms
Enviado/Recibido: Mensaje 4 | RTT: 0.266 ms
Traceback (most recent call last):
  File "C:\Users\valen\OneDrive\Escritorio\Documentos\Facultad\4to año 2do cutrimestre\Sistemas Distribuidos Y Paralelos\tp2\cliente_tcp.py", line 43, in <module>
    time.sleep(2) # Pausa entre intercambios, fuera de la medición del RTT
    ~~~~~^~~~~~
KeyboardInterrupt
```